



We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech



SQL BASICS

DAY- MONTH - YEAR

Contents

01

SQL Function

02

Set Operations

03

**Nested Sub
Queries**

SQL Functions



SQL Functions

Introduction

- In SQL, a **function** is a **predefined operation** that takes **one or more inputs** (arguments), performs a **specific task** (like calculation, formatting, or aggregation), and **returns a value**.
- Functions in SQL help you manipulate data, perform calculations, transform text, work with dates, and more — **without having to write long repetitive code**.
- **Types of SQL Function:**

Type of Function	Description
Single-row functions	Process each row individually and return one result per row.
Multiple-row functions (also called Group functions or Aggregate functions)	Process groups of rows and return one result for the group.

SQL Functions

Single-Row Functions

- **Single –Row Functions operate on each row** of the query result and **return one result per row**.
- This function used to **modify or retrieve specific values** from **each row individually**.
- **Categories include:**

Category	Purpose	Examples
String Functions	Work with text values	UPPER(), LOWER(), LENGTH(), SUBSTRING()
Numeric Functions	Work with numbers	ROUND(), ABS(), MOD()
Date & Time Functions	Work with dates/times	NOW(), CURDATE(), DATEDIFF()
Conversion Functions	Convert data from one type to another	CAST(), CONVERT()
General Functions	Return system information or handle NULL values	IFNULL(), COALESCE(), NULLIF()
Conditional Functions	Perform logic-based operations	IF(), CASE

SQL Functions

Single-Row Functions : String Functions

Function	Description	Example
CONCAT()	Joins strings	SELECT CONCAT('Hello', ' World'); → <i>Hello World</i>
LENGTH()	Returns length of string in bytes	SELECT LENGTH('Hello'); → 5
LOWER() / UPPER()	Convert case	SELECT UPPER('hello'); → <i>HELLO</i>
TRIM()	Remove spaces	SELECT TRIM(' hi '); → <i>hi</i>
SUBSTRING()	Extract part of string	SELECT SUBSTRING('Database', 5); → <i>base</i>

SQL Functions

Single-Row Functions : Numeric Functions

Function	Description	Example
ABS()	Absolute value	SELECT ABS(-5); → 5
ROUND()	Round number	SELECT ROUND(3.567, 2); → 3.57
CEIL() / FLOOR()	Round up/down	SELECT CEIL(4.2); → 5
MOD()	Remainder	SELECT MOD(10, 3); → 1
POWER()	Raise to power	SELECT POWER(2, 3); → 8

SQL Functions

Single-Row Functions : Date & Time Functions

Function	Description	Example
NOW()	Current date & time	SELECT NOW(); → 2025-08-12 15:32:00
CURDATE() / CURTIME()	Current date / time	SELECT CURDATE(); → 2025-08-12
YEAR(), MONTH(), DAY()	Extract parts	SELECT YEAR(NOW()); → 2025
DATE_ADD()	Add time interval	SELECT DATE_ADD('2025-08-12', INTERVAL 5 DAY); → 2025-08-17
DATEDIFF()	Days between dates	SELECT DATEDIFF('2025-08-20', '2025-08-12'); → 8

SQL Functions

Single-Row Functions : Conversion Functions

Function	Description	Example
CAST(expression AS data_type)	Converts a value from one data type to another	CAST('2025-08-12' AS DATE) → 2025-08-12
CONVERT(expression, data_type)	Similar to CAST(), changes a value to the specified data type	CONVERT('123', UNSIGNED) → 123
FORMAT(number, decimal_places)	Formats a number to a specified decimal places, returns as a string	FORMAT(12345.6789, 2) → 12,345.68
BINARY	Forces a string comparison to be case-sensitive	BINARY 'Hello' = 'hello' → 0 (false)

SQL Functions

Single-Row Functions : Conversion Functions

Function	Description	Example
CHAR()	Converts integer ASCII values to characters	CHAR(65) → A
CAST(... USING charset)	CAST(... USING charset)	CAST(... USING charset)

SQL Functions

Single-Row Functions : Conversion Functions

Function	Description	Example
CHAR()	Converts integer ASCII values to characters	CHAR(65) → A
CAST(... USING charset)	CAST(... USING charset)	CAST(... USING charset)

SQL Functions

Single-Row Functions : General Functions

Function	Description	Example
IF(condition, value_if_true, value_if_false)	Returns one value if condition is true, otherwise another value	IF(5 > 3, 'Yes', 'No') → Yes
IFNULL(expression, alt_value)	Returns alt_value if expression is NULL, else returns expression	IFNULL(NULL, 'N/A') → N/A
NULLIF(expr1, expr2)	Returns NULL if both expressions are equal, else returns expr1	NULLIF(5, 5) → NULL
COALESCE(expr1, expr2, ...)	Returns the first non-NULL value from a list	COALESCE(NULL, NULL, 'Hello') → Hello
VERSION()	Returns the MySQL server version	VERSION() → 8.0.34
DATABASE()	Returns the current database name	DATABASE() → sales_db
USER()	Returns the current MySQL user	USER() → root@localhost

SQL Functions

Single-Row Functions : Conditional Functions

Function	Description	Example
IF(condition, value_if_true, value_if_false)	Returns one value if the condition is true, otherwise another value	IF(10 > 5, 'Yes', 'No') → Yes
CASE	Checks conditions in sequence and returns the first matching result	CASE WHEN score >= 90 THEN 'A' WHEN score >= 75 THEN 'B' ELSE 'C' END
IFNULL(expression, alt_value)	Returns alt_value if expression is NULL, else returns the expression	IFNULL(NULL, 'N/A') → N/A
NULLIF(expr1, expr2)	Returns NULL if both expressions are equal, else returns expr1	NULLIF(100, 100) → NULL
COALESCE(expr1, expr2, ...)	Returns the first non-NULL value from the list	COALESCE(NULL, NULL, 'Data') → Data

SQL Functions

Multiple-Row Functions (Group Functions / Aggregate Functions)

- **Multiple –Row Functions** operate a set of rows and return a single result for the entire set.
- This function **used for summarizing or aggregating data across multiple rows.**
- **Categories include:** Consider the table - **Student(StudentID, Name, Email, DOB, Major)**

Function	Description	Example Query	Example Output
COUNT()	Counts the number of rows (or non-NULL values in a column).	SELECT COUNT(*) AS total_students FROM Student;	50
SUM()	Adds up all numeric values in a column. <i>(Assume a Credits column)</i>	SELECT SUM(Credits) AS total_credits FROM Student;	300
AVG()	Calculates the average of numeric values. <i>(Assume GPA column)</i>	SELECT AVG(GPA) AS avg_gpa FROM Student;	3.4

SQL Functions

Multiple-Row Functions (Group Functions / Aggregate Functions)

Function	Description	Example Query	Example Output
MIN()	Returns the smallest value in a column.	SELECT MIN(DOB) AS oldest_student FROM Student;	1998-02-15
MAX()	Returns the largest value in a column.	SELECT MAX(DOB) AS youngest_student FROM Student;	2005-07-21
GROUP_CONCAT()	Concatenates values from multiple rows into a single string.	SELECT GROUP_CONCAT(Name) AS all_names FROM Student;	John, Mary, Alex, Priya
STDDEV()	Calculates the standard deviation of numeric values. (Assume GPA column)	SELECT STDDEV(GPA) AS gpa_stddev FROM Student;	0.45

SQL Functions

Multiple-Row Functions (Group Functions / Aggregate Functions)

Function	Description	Example Query	Example Output
VARIANCE()	Calculates the variance of numeric values. (<i>Assume GPA column</i>)	SELECT VARIANCE(GPA) AS gpa_variance FROM Student;	0.20

SQL Functions

Analytic Functions(Window Functions)

- Analytic Functions are **SQL functions** that perform calculations **over a group of rows (a window)** while **still returning a value** for **every individual row**.
- They **do not collapse** data like **GROUP BY**. They keep **all rows** and **add analytical information**.
- These functions are highly useful for **tasks such as ranking, calculating running totals, and finding moving averages** while still retaining individual row details.

SQL Functions

Analytic Functions(Window Functions)

- Analytic functions – **Syntax**

```
window_function_name([expression]) OVER ( [PARTITION BY expression] [ORDER BY expression  
[ASC|DESC]] [ROWS or RANGE frame_clause]
```

- **window_function_name**: which is nothing but the name of your window function
- **PARTITION BY**: Result set is divided into partitions, and then the window function is applied.
- **ORDER BY**: This specifies the order of the rows within each partition.
- **ROWS** or **RANGE** : Defines the window of rows that should be considered for the calculation

SQL Functions

Analytic Functions(Window Functions)

- Analytic Functions include:
 - **Ranking Functions** → ROW_NUMBER, RANK, DENSE_RANK
 - **Navigation Functions** → LAG, LEAD
 - **Aggregate Window Functions** → SUM, AVG, COUNT with OVER()
 - **Value Functions** → FIRST_VALUE, LAST_VALUE

SQL Functions

Analytic Functions(Window Functions)

1. **ROW_NUMBER:** This function is used to assigns a unique sequential integer to rows within a partition

Example:

```
SELECT
    id,
    department,
    salary,
    ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num
FROM employee
```

Output:

id	department	salary	row_num
1003	Finance	50000	1
1004	Finance	50000	2
1005	HR	75000	1
1010	HR	75000	2
1002	HR	45000	3
1007	HR	45000	4
1008	IT	50000	1
1009	IT	50000	2
1001	IT	35000	3
1006	IT	35000	4

SQL Functions

Analytic Functions(Window Functions)

2. RANK and DENSE_RANK : Both are **ranking window functions** used to **assign positions** based on a **given order**.

Example:

```
SELECT  
    id,  
    department,  
    salary,  
    RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS EMP_RANK,  
    DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS  
    EMP_DENSE_RANK  
FROM employee;
```

id	department	salary	EMP_RANK	EMP_DENSE_RANK
1003	Finance	50000	1	1
1004	Finance	50000	1	1
1005	HR	75000	1	1
1010	HR	75000	1	1
1002	HR	45000	3	2
1007	HR	45000	3	2
1008	IT	50000	1	1
1009	IT	50000	1	1
1001	IT	35000	3	2
1006	IT	35000	3	2

SQL Functions

Analytic Functions(Window Functions)

3. SUM and AVG: Calculate the sum of the columns within a window, and calculate the average of the columns within a window.

Example:

```
SELECT
  id,
  department,
  salary,
  SUM(salary) OVER (PARTITION BY department) AS total_salary,
  AVG(salary) OVER (PARTITION BY department) AS avg_salary
FROM employee;
```

id	department	salary	total_salary	avg_salary
1003	Finance	50000	100000	50000
1004	Finance	50000	100000	50000
1002	HR	45000	240000	60000
1005	HR	75000	240000	60000
1007	HR	45000	240000	60000
1010	HR	75000	240000	60000
1001	IT	35000	170000	42500
1006	IT	35000	170000	42500
1008	IT	50000	170000	42500
1009	IT	50000	170000	42500

Quiz



1) Which of the following is a Single-Row String Function in MySQL?

a) SUM()

b) LENGTH()

c) AVG()

d) COUNT()

Answer : Option b)

Quiz



2) What will be the result of the query `SELECT ROUND(123.456, 2);`

a) 123

b) 123.4

c) 123.46

d) 124

Answer : Option c)

Quiz



3) Which of the following functions combines multiple row values into a single comma-separated string in MySQL?

a) `CONCAT()`

b) `GROUP_CONCAT()`

c) `STRING_AGG()`

d) `CONCAT_WS`

Answer : Option b)

Quiz



4) Which function is used to find the average of values in a numeric column?

a) MEADIAN()

b) MEAN()

c) AVG()

d) AVERAGE()

Answer : Option c)

Quiz



5) Which of the following statements is correct about `COUNT(column_name)`?

a) It counts all rows, including NULL values

b) It counts only rows where `column_name` is NOT NULL

c) It counts only rows where `column_name` is NULL

d) It sums up column values

Answer : Option b)

Set Operations



Set Operations

Overview

- SQL provides **set operators** to **compare rows** from **two or more tables** or to combine the results obtained from **two or more queries** to obtain the result.
- These operators are used to **join the results** of two or more SELECT statements.
- We can use **set operators** that **list the results** from the given tables to a single result or in different rows.
- SQL supports set operators, which can be **performed on the data**.
- These operators are used to get the **desired results** from the **table data stored in the table**.
- The set operators look like the **SQL joins**, but there is a **big difference** where SQL joins combine the columns from different tables, whereas **SQL operators combine rows from different queries**.

Set Operations

Overview

Syntax:

```
SELECT first_select_query
```

```
set_operator
```

```
SELECT second_select_query
```

Rules for Set Operators in SQL

1. Number of Columns Must Be the Same in both select queries
2. Column Data Types Must Be Compatible in both select queries
3. Column Names Need Not Be the Same in both tables of select queries

Set Operations

Types of Set Operations

- There are different types of set operators that are mentioned below:
 - **UNION**
 - **UNION ALL**
 - **MINUS**
 - **INTERSECT**

Note :

- While MySQL directly supports UNION and UNION ALL, it does not support MINUS and INTERSECT directly.
- In MySQL, you can achieve the functionality of the INTERSECT operator using an INNER JOIN or a EXISTS clause and Minus/Except using LEFT JOIN or NOT EXISTS

Set Operations

Types of Set Operations

- Let us look into each set operator in more detail with the **Employees_US** and **Employees_UK** table as given below

emp_id	emp_name	department
1	Alice	HR
2	Bob	IT
3	Carol	Marketing

emp_id	emp_name	department
2	Bob	IT
4	Dave	HR
5	Eve	Finance

Set Operations

Union

- UNION combines the **results of two or more SELECT statements**.
- To successfully execute the operation of UNION, the **number of columns and the data type** must be the **same in both tables**.
- After performing the UNION operation, the **duplicate rows** will be **eliminated from the results**.

Syntax:

```
SELECT expression_1, expression_2, ... , expression_n FROM table_1
```

```
UNION
```

```
SELECT expression_1, expression_2, ... , expression_n FROM table_2
```

Set Operations

Union

-- Example: The row (2, Bob, IT) appears in both tables but is included only once because **UNION** removes duplicates

```
SELECT emp_id, emp_name, department FROM Employees_US
```

```
UNION
```

```
SELECT emp_id, emp_name, department FROM Employees_UK;
```

OUTPUT

emp_id	emp_name	department
1	Alice	HR
2	Bob	IT
3	Carol	Marketing
4	Dave	HR
5	Eve	Finance

Set Operations

Union All

- UNION ALL is also used **to combine the results of two or more SELECT statements**.
- UNION and UNION ALL are similar in their functioning, but there is a slight difference.
- To successfully execute the operation of Union All, the **number of columns and the data type must be the same in both tables**.
- After performing the UNION ALL operation, the **duplicate rows will not be eliminated from the results**, and all the data is displayed in the result without removing the duplication.

Syntax:

```
SELECT expression_1, expression_2, ... , expression_n FROM table_1
```

```
UNION ALL
```

```
SELECT expression_1, expression_2, ... , expression_n FROM table_2
```

Set Operations

Union All

Example: The row (2, Bob, IT) appears twice because UNION ALL does not remove duplicates

```
SELECT emp_id, emp_name, department FROM Employees_US
```

```
UNION ALL
```

```
SELECT emp_id, emp_name, department FROM Employees_UK;
```

OUTPUT

emp_id	emp_name	department
1	Alice	HR
2	Bob	IT
3	Carol	Marketing
2	Bob	IT
4	Dave	HR
5	Eve	Finance

Set Operations

Quiz



1. Which set operator in MySQL keeps duplicate rows from both queries?

a) UNION

b) UNION ALL

c) INTERSECT

d) EXCEPT

Answer : Option b)

Set Operations

Quiz



2. Which of the following is true about set operators?

a) All queries must have the same number of columns

b) The data types of corresponding columns must be compatible

c) ORDER BY can be applied only after the final set operator

d) All the Above

Answer : Option d)

Subqueries



Subqueries

Introduction

- **Subqueries** also called **nested query** or **inner query** is a query which is present inside another SQL query.
- In a subquery, the **outer query's result** is **dependent** on the **result set of the inner query** so it will also be called a **nested query**.
- Subqueries must be **enclosed within parenthesis**.

Syntax:

```
( SELECT [DISTINCT] subquery_select_parameter  
  FROM {table_name | view_name}, {table_name | view_name} ...  
  [WHERE search_conditions]  
  [GROUP BY column_name [,column_name ] ...]  
  [HAVING search_conditions] )
```


Subqueries

Introduction

- Let us understand the subqueries using Course and Prerequisite tables.

Course table:

	CourseID	Title	Credits	DepartmentID
▶	CS101	Data Structures	4	D01
	CS102	DBMS	3	D01
	CS103	Operating Systems	3	D01
	EC101	Digital Circuits	3	D02
	EC102	Analog Electronics	3	D02
	IT101	C Programming	5	D03
	IT102	Problem Solving	2	D03

Prerequisite table:

	CourseID	PrerequisiteID
▶	CS102	CS101
	CS103	CS101
	EC102	EC101

Department table:

	DepartmentID	Name	Building
▶	D01	Computer Science	CS BLOCK
	D02	Electronics & Communication	ECE BLOCK
	D03	Information Technology	IT BLOCK

Subqueries

Types of Subqueries : Single Row Subquery

- Subqueries that **return a single row** as an **output to their parent query** is called **single-row subqueries**.
- The **inner query** is **independent** of the outer query and gives a meaningful result by running on its own.
- **Single-row subqueries** are used in a SQL **SELECT** statement with a **HAVING clause, WHERE clause, or a FROM clause and a comparison operator**.
- Single-row subqueries can also be used with the **WHERE clause in the SELECT statement** to filter the results of the outer query.

Subqueries

Example

Example #1: Minimum credits from the course table

OUTPUT

```
SELECT * FROM Course
```

```
WHERE Credits =
```

```
(SELECT MIN(Credits) FROM Course );
```

	CourseID	Title	Credits	DepartmentID
▶	IT102	Problem Solving	2	D03

Explanation:

- The subquery is executed **first which selects minimum credits** from the course table.
- Then this result is passed on to the **WHERE clause of the outer query**.
- Then the **outer query is executed** which selects rows where credits is equal to the **result of the subquery and returns the final result**.

Subqueries

Types of Subqueries : Multiple Row Subquery

- **Subqueries** that **return multiple rows** as an **output** to **their parent query** are called **multiple-row subqueries**.
- Multiple row subqueries can be used in a **SQL SELECT** statement with a **HAVING** clause, **WHERE** clause, a **FROM** clause, and a logical operator(**ALL**, **IN**, **NOT IN**, and **ANY**).

Example #2: Find Courses which has prerequisite.

```
SELECT CourseID, Title FROM Course WHERE
CourseID IN (SELECT CourseID FROM prerequisite);
```

OUTPUT

	CourseID	Title
▶	CS102	DBMS
	CS103	Operating Systems
	EC102	Analog Electronics

Subqueries

Types of Subqueries : Multiple Column Subqueries

- Subqueries that return multiple columns as an output to their parent query are called multiple-column subqueries.

Example #3: Retrieve the multiple column.

```
SELECT CourseID, Title FROM Course
WHERE (CourseID, DepartmentID) =
(SELECT CourseID, DepartmentID FROM Course WHERE Credits=2);
```

OUTPUT

	CourseID	Title
▶	IT102	Problem Solving

Example #4: Retrieve the multiple column.

```
SELECT CourseID, Title FROM Course
WHERE (CourseID, DepartmentID) IN
(SELECT CourseID, DepartmentID FROM Course WHERE Credits=3);
```

OUTPUT

	CourseID	Title
▶	CS102	DBMS
	CS103	Operating Systems
	EC101	Digital Circuits
	EC102	Analog Electronics

Subqueries

Types of Subqueries : Correlated Subqueries

- **Subqueries** that return **multiple columns** as **output** depending on the **information** obtained from the **parent query** are called **correlated subqueries**.
- In simple, A **subquery** that refers **to the outer query** for **each row processed**.
- The **interdependence** of the inner and outer query makes it complicated to understand.
- **Correlated subqueries** can be used in **SELECT** statements using **WHERE** and **FROM** clauses.
- In the below **Example#5**, The subquery calculates the **average credit for each department**, using **C1.DepartmentID = C2.DepartmentID** and the **outer query compares each course's credit against this average**.

Subqueries

Types of Subqueries : Correlated Subqueries

Example #5: Retrieve the CourseID, Title, and Credits of all courses whose number of credits is greater than the average credits of all courses in the same department.

```
SELECT CourseID, Title, Credits  
FROM Course C1  
WHERE Credits >  
(SELECT AVG(Credits) FROM Course C2  
WHERE C1.DepartmentID = C2.DepartmentID);
```

OUTPUT

	CourseID	Title	Credits
▶	CS101	Data Structures	4
	IT101	C Programming	5

Subqueries

Types of Subqueries : Nested Subqueries

- **Subqueries** that are **inside another subquery** are called **nested subqueries**.
- Subqueries are **executed level by level**.
- The **innermost** is **executed first**, and then the **outer ones**.

Example #6: List all prerequisites of the courses offered by the department located in the 'CS BLOCK'.

```
SELECT * FROM prerequisite WHERE CourseID IN  
( SELECT CourseID FROM Course WHERE DepartmentID =  
(SELECT DepartmentID FROM Department WHERE Building = 'CS BLOCK'));
```

OUTPUT

	CourseID	PrerequisiteID
▶	CS102	CS101
	CS103	CS101

Subqueries

Types of Subqueries : Nested Subqueries

Explanation

- The **innermost subquery** finds the DepartmentID of the department that is located in the building named 'CS BLOCK'.
- The **middle subquery** uses the DepartmentID from the inner query and returns CourseIDs that belong to the department in the 'CS BLOCK'.
- The **outer query** fetches all records where the CourseID matches any of the course IDs offered by the department in the 'CS BLOCK'.

Quiz



1. Which operator is most commonly used with a single-row subquery?

a) IN

b) =

c) ANY

d) ALL

Answer : Option b)

Quiz



2. Which operator is used with multiple-row subqueries?

a) IN

b) =

c) >

d) < >

Answer : Option a)

Quiz



3. Multiple column subqueries return:

a) Only one column

b) Multiple columns but only one row

c) Multiple columns and multiple rows

d) Only aggregated results

Answer : Option c)

Quiz



4. Which best defines a correlated subquery?

a) A subquery that runs independently of the outer query

b) A subquery that references columns of the outer query

c) A subquery with only aggregate functions

d) A subquery that always returns multiple rows

Answer : Option b)

Quiz



5. Which of the following is true about nested subqueries?

a) They can be used in **SELECT, INSERT, UPDATE, and DELETE** statements

b) They only work in **SELECT** statements

c) They always slow down the query and should be avoided

d) They do not return any result

Answer : Option a)

THANK YOU